Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas





Título del Proyecto:

CLOUDGRID DATACENTER

FASE 2: GLOBAL NETWORK ORCHESTRATOR

PONDERACIÓN: 25

Horas Aproximadas: 50

Índice

1. Resumen Ejecutivo **3**

2. Competencia que desarrollaremos **3**

3. Objetivos del Aprendizaje **4**

3.1 Objetivo General 4
3.2 Objetivos Específicos 4

4. Enunciado del Proyecto **5**

4.1 Descripción del problema a resolver 5
4.2 Alcance del proyecto 13

5. Desarrollo de Habilidades Blandas **13**

5.1 Proyectos Individuales 13
5.2 Resumen de Puntuaciones 15
5.3 Observaciones 16

1. Resumen Ejecutivo

Tras consolidar en la Fase 1 la infraestructura física y lógica de un único centro de datos, CloudGrid DataCenter inicia su expansión regional: ahora opera una red de Data Centers interconectados por enlaces de fibra óptica en América y Europa. Esta expansión trae consigo nuevos desafíos de ingeniería: resolver nombres de host a direcciones IP en tiempo constante, decidir qué trabajo de infraestructura se ejecuta primero cuando decenas compiten por los mismos recursos, registrar cuánto tráfico fluye entre cada par de sedes y determinar por dónde viajan los datos —y qué sedes quedan aisladas— cuando un enlace falla.

La Fase 2, denominada CloudGrid Global Network Orchestrator, extiende la solución en C++ nativo incorporando cuatro nuevas estructuras de datos: una Tabla Hash con direccionamiento abierto y doble dispersión para el Servidor DNS interno, una Matriz Dispersa ortogonal para la Matriz de Tráfico entre Data Centers (origen × destino), un Montículo Binario Mínimo (Min-Heap) para el Planificador de Trabajos por prioridad y un Grafo no dirigido y ponderado, representado con listas de adyacencia, que modela la Red Backbone y se analiza mediante los recorridos en anchura (BFS) y en profundidad (DFS). Las cuatro estructuras colaboran en un mismo flujo: cada trabajo despachado desde el Heap viaja por la ruta que encuentra el BFS, su volumen se acumula en la Matriz Dispersa y los nuevos servicios quedan publicados en la Tabla Hash.

2. Competencia que desarrollaremos

A través de la segunda fase del sistema CloudGrid DataCenter, se consolidarán competencias en el diseño, análisis e implementación de estructuras de datos de acceso directo, jerárquicas y de red en C++, sin el uso de la librería de plantillas estándar (STL) para las estructuras evaluadas. El estudiante comprenderá por qué una función hash bien diseñada permite búsquedas en tiempo O(1) promedio, cómo un montículo binario almacenado en un arreglo garantiza inserciones y extracciones en O(log n), cuándo conviene una matriz dispersa frente a una matriz convencional y cómo los recorridos BFS y DFS revelan propiedades de una red como el número mínimo de saltos, la alcanzabilidad y las componentes conexas.

Adicionalmente, se fortalecerán capacidades de análisis de complejidad algorítmica y de toma de decisiones de diseño (tamaño de la tabla, factor de carga, estrategia de colisiones, representación del grafo), integración de módulos que comparten información entre sí, procesamiento de archivos CSV de carga masiva y generación automatizada de diagramas con Graphviz. Todo esto fomenta el pensamiento analítico, la abstracción de problemas reales de redes y computación en la nube, y el desarrollo de código limpio, mantenible y profesional.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Extender la plataforma CloudGrid DataCenter en C++ hacia una red multi-región de centros de datos, implementando desde cero una Tabla Hash, una Matriz Dispersa ortogonal, un Montículo Binario (Heap) y un Grafo con sus recorridos BFS y DFS, integrados en un flujo coherente de operación y acompañados de reportes visuales generados con Graphviz.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- Implementar una Tabla Hash con direccionamiento abierto y doble dispersión, incluyendo el manejo de colisiones, marcas de borrado (lápidas) y rehashing automático al superar el factor de carga máximo.
- Diseñar e implementar una Matriz Dispersa ortogonal que registre el tráfico acumulado entre pares de Data Centers, almacenando únicamente las celdas que contienen información.
- Construir un Montículo Binario Mínimo sobre un arreglo dinámico para planificar trabajos de infraestructura por prioridad, aplicando inserción (sift-up), extracción del mínimo (sift-down), cambio de prioridad y construcción en O(n) (heapify).
- Modelar la red backbone como un grafo no dirigido y ponderado mediante listas de adyacencia, y aplicar los recorridos BFS y DFS para obtener rutas de menor número de saltos, alcanzabilidad y componentes conexas.
- Analizar y justificar la complejidad temporal y espacial de cada operación implementada, comparando las representaciones utilizadas con sus alternativas (por ejemplo, matriz de adyacencia frente a lista de adyacencia).
- Integrar Graphviz para traducir dinámicamente el estado de las cuatro estructuras a imágenes PNG/SVG, y documentar el diseño en un manual técnico versionado con Git/GitHub.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

Arquitectura y Mapeo de Estructuras de la Fase 2

El núcleo de CloudGrid Global Network Orchestrator se fundamenta en la interoperabilidad de las siguientes cuatro estructuras de datos, cada una con un rol técnico específico y no intercambiable:

Estructura	Entidad Modelada	Clave / Índice	Justificación Técnica y Operaciones
Tabla Hash (Direcciona-miento abierto, doble dispersión)	Servidor DNS Interno (Hostname → IP)	Hostname, convertido a entero con hash polinomial	Resuelve nombres de host en O(1) promedio. Tamaño primo, funciones h1 y h2 para el sondeo, lápidas en las eliminaciones y rehashing al siguiente primo ≥ 2m cuando α > 0.70.
Matriz Dispersa Ortogonal	Matriz de Tráfico entre Data Centers	Fila: DC Origen / Columna: DC Destino	Sólo almacena pares origen-destino con tráfico real. Cabeceras ordenadas; cada nodo se enlaza arriba, abajo, izquierda y derecha. Acumula GB transferidos y número de trabajos.
Montículo Binario Mínimo (Min-Heap)	Planificador de Trabajos (Job Scheduler)	Prioridad (1 = crítica); en empate, orden de llegada	Implementado sobre un arreglo dinámico. Inserción con sift-up, extracción del mínimo con sift-down, cambio de prioridad y construcción heapify O(n) desde CSV.
Grafo No Dirigido Ponderado (Lista de Adyacencia)	Red Backbone de Data Centers	Código de DC (vértice); latencia en ms (arista)	Modela sedes y enlaces de fibra. BFS: niveles y ruta de menor número de saltos. DFS: alcanzabilidad, componentes conexas y verificación de conectividad ante fallas.

El sistema conserva los dos perfiles de usuario de la Fase 1: **Administrador de Data Center** y **Usuario DevOps / Cliente Corporativo**. Antes de detallar las funcionalidades de cada perfil, se describe el flujo integrado que une a las cuatro estructuras:

Flujo Integrado de Operación de un Trabajo

1. Solicitud: un cliente solicita un trabajo (DEPLOY, MIGRAR, REPLICAR, BACKUP o PARCHE) indicando DC origen, DC destino, volumen en GB y prioridad; el trabajo se inserta en el Min-Heap.

2. Despacho: el administrador despacha el siguiente trabajo; se extrae la raíz del Heap (el de menor valor de prioridad).

3. Enrutamiento: se ejecuta un BFS sobre el grafo desde el DC origen para obtener la ruta con menor número de saltos hacia el DC destino. Si no existe ruta (por ejemplo, por un enlace caído), el trabajo se marca como BLOQUEADO y se reporta.

4. Registro de tráfico: al completarse, el volumen transferido se acumula en la celda [origen][destino] de la Matriz Dispersa, creando la celda y sus cabeceras si no existían.

5. Publicación DNS: si el trabajo es un DEPLOY, el hostname del nuevo servicio y su IP se insertan en la Tabla Hash, quedando disponibles para su resolución inmediata.

Funciones de Administrador

El administrador del sistema contará con un menú principal dentro de una interfaz gráfica formal e intuitiva que le permitirá gestionar integralmente la red global de Data Centers a través de las siguientes funcionalidades clave:

1. Gestión de la Red Backbone (Grafo)

Permite registrar y eliminar Data Centers (Código, Ciudad, País, Capacidad en Racks) como vértices del grafo, así como crear, modificar y eliminar enlaces de fibra óptica (DC A, DC B, Latencia en ms, Ancho de Banda en Gbps) como aristas no dirigidas y ponderadas. El grafo se representa mediante listas de adyacencia implementadas manualmente. Incluye la opción Simular Falla de Enlace, que desactiva temporalmente una arista sin eliminarla, para estudiar su impacto en la conectividad de la red.

2. Carga Masiva de Datos (CSV):

Procesa archivos CSV para poblar en lote las cuatro estructuras: la red backbone (Grafo), los registros DNS (Tabla Hash), los trabajos pendientes (Min-Heap) y el historial de tráfico (Matriz Dispersa). El analizador sintáctico (parser) valida tipos de datos y referencias (por ejemplo, que los DC de un trabajo existan en el grafo), reportando las líneas con error sin detener la carga. Los trabajos cargados masivamente deben incorporarse al Heap mediante el algoritmo heapify (construcción ascendente en O(n)) y no mediante inserciones sucesivas.

3. Servidor DNS Interno (Tabla Hash):

Administra los registros DNS (Hostname, Dirección IP, Tipo de Registro A/CNAME, TTL en segundos, Código de Cliente). La tabla inicia con tamaño primo m = 13 y utiliza direccionamiento abierto con doble dispersión: h1(k) = k mod m y h2(k) = 1 + (k mod (m − 2)), donde k es el entero obtenido al aplicar una función hash polinomial al hostname. Las eliminaciones dejan una marca de BORRADO (lápida) para no romper

las cadenas de sondeo. Cuando el factor de carga α = n/m supera 0.70, la tabla se redimensiona al siguiente número primo mayor o igual a 2m y se reinsertan todos los registros (rehashing). El módulo muestra estadísticas: tamaño, registros, factor de carga, colisiones totales y promedio de sondeos por búsqueda.

4. Planificador de Trabajos por Prioridad (Min-Heap):

Gestiona la cola de trabajos de infraestructura (ID Trabajo, Tipo, DC Origen, DC Destino, Volumen GB, Prioridad 1–10, Código de Cliente, Hostname). La raíz del Montículo Binario Mínimo contiene siempre el trabajo más urgente; en caso de empate de prioridad se atiende primero el de menor número de llegada. El administrador puede encolar trabajos, despachar el siguiente (extracción de la raíz con reacomodo sift-down), modificar la prioridad de un trabajo en cola (escalamiento) restaurando la propiedad de heap, y cancelar trabajos. Cada trabajo despachado ejecuta el flujo integrado descrito anteriormente.

5. Análisis de Conectividad y Rutas (BFS y DFS):

Implementa ambos recorridos sobre el grafo utilizando colas y pilas propias. El BFS desde un DC origen debe mostrar el orden de visita y el nivel (número de saltos) de cada Data Center, y reconstruir la ruta de menor número de saltos hacia un destino mediante el arreglo de predecesores, indicando la latencia acumulada de dicha ruta. El DFS debe mostrar el orden de visita, identificar las componentes conexas de la red y verificar si la red sigue completamente conectada tras simular la falla de un enlace, listando los Data Centers que quedarían aislados. Nota: la ruta de menor número de saltos no siempre es la de menor latencia; el estudiante deberá explicar esta diferencia en su manual técnico.

6. Matriz de Tráfico entre Data Centers (Matriz Dispersa):

Registra el tráfico acumulado entre cada par origen-destino mediante una Matriz Dispersa ortogonal cuyas cabeceras de filas (DC origen) y columnas (DC destino) se crean dinámicamente y se mantienen ordenadas alfabéticamente por código. Cada celda almacena el volumen total en GB y el número de trabajos completados. El módulo permite consultar una celda, recorrer una fila (todo lo que envía un DC), recorrer una columna (todo lo que recibe un DC), eliminar registros, obtener el top 3 de pares origen-destino con mayor tráfico y mostrar el porcentaje de ocupación de la matriz (celdas ocupadas / celdas posibles).

7. Módulo de Visualización Graphviz

Genera y exporta dinámicamente la representación visual en tiempo real de las cuatro estructuras de la fase (Tabla Hash, Matriz Dispersa, Min-Heap y Grafo) mediante la generación automatizada de archivos de descripción DOT y su compilación a imágenes en formatos PNG/SVG utilizando Graphviz.

Funciones del Módulo Usuario DevOps / Cliente

Los usuarios del módulo DevOps y Clientes Corporativos disponen de un entorno dedicado con las siguientes funcionalidades para operar sus servicios sobre la red global:

1. Iniciar Sesión

Módulo de autenticación que valida las credenciales del usuario (correo electrónico y contraseña) garantizando un acceso restringido y seguro a su perfil corporativo.

2. Resolución de Nombres (DNS)

Permite resolver un hostname a su dirección IP consultando la Tabla Hash, mostrando el índice final donde se encontró el registro y la cantidad de sondeos realizados. El cliente también puede registrar, actualizar o eliminar los hostnames asociados a su código de cliente.

3. Solicitud y Seguimiento de Trabajos

Permite solicitar nuevos trabajos indicando tipo, DC origen, DC destino, volumen y prioridad (los clientes con plan básico de la Fase 1 no pueden solicitar una prioridad menor a 3). El cliente puede consultar el estado de sus trabajos (EN COLA, COMPLETADO, BLOQUEADO), su posición actual dentro del arreglo del heap y cuántos trabajos más urgentes existen antes del suyo.

4. Consulta de Rutas y Alcance de Red

Permite consultar la ruta de menor número de saltos entre dos Data Centers (BFS), mostrando la secuencia de sedes, el número de saltos y la latencia acumulada, así como explorar todos los Data Centers alcanzables desde una sede determinada (DFS) en el orden en que fueron visitados.

5. Consulta de Tráfico de su Infraestructura:

Permite consultar en la Matriz Dispersa el tráfico generado entre los Data Centers involucrados en sus trabajos, ya sea una celda específica origen-destino o el resumen por fila (envíos) y columna (recepciones).

Especificación de Carga Masiva de Datos (Archivos CSV)

El sistema debe incorporar un módulo robusto de importación mediante archivos planos delimitados por comas (.csv) para agilizar la puesta en marcha de la red global:

Formato de Archivo: Red_Backbone.csv (Alimenta el Grafo)

Estructura de columnas: DC_Origen, Ciudad_Origen, DC_Destino, Ciudad_Destino, Latencia_ms, AnchoBanda_Gbps (los Data Centers que no existan se crean automáticamente como vértices)

Ejemplo:
DC-GUA,Guatemala,DC-MEX,Ciudad de México,18,100
DC-GUA,Guatemala,DC-MIA,Miami,24,400
DC-MIA,Miami,DC-MAD,Madrid,95,200

Formato de Archivo: Registros_DNS.csv (Alimenta la Tabla Hash)

Estructura de columnas: Hostname, Direccion_IP, Tipo_Registro, TTL_Segundos, Codigo_Cliente

Ejemplo:
api.cloudgrid.gt,10.0.1.40,A,3600,CLI001
auth.cloudgrid.gt,10.0.1.20,A,3600,CLI002
www.cloudgrid.gt,api.cloudgrid.gt,CNAME,600,CLI001

Formato de Archivo: Trabajos.csv (Alimenta el Min-Heap mediante heapify)

Estructura de columnas: ID_Trabajo, Tipo, DC_Origen, DC_Destino, Volumen_GB, Prioridad, Codigo_Cliente, Hostname

Ejemplo:
101,MIGRAR,DC-GUA,DC-MIA,120,2,CLI001,api.cloudgrid.gt
102,DEPLOY,DC-MEX,DC-DFW,40,5,CLI002,auth-v2.cloudgrid.gt
104,BACKUP,DC-BOG,DC-GRU,510,1,CLI003,db-master.cloudgrid.gt

Formato de Archivo: Historial_Trafico.csv (Alimenta la Matriz Dispersa)

Estructura de columnas: DC_Origen, DC_Destino, Volumen_GB, Cantidad_Trabajos

Ejemplo:
DC-GUA,DC-MIA,120,3
DC-MEX,DC-DFW,300,2
DC-BOG,DC-GRU,510,4

Área de Reportes

Cada estructura de datos debe contar con un método dedicado capaz de exportar su estado actual a un archivo en formato DOT y compilarlo a una imagen PNG/SVG mediante llamadas al comando nativo 'dot -Tpng <archivo>.dot -o <imagen>.png'. La interfaz gráfica debe renderizar estos diagramas directamente en pantalla:

Reporte 1: Tabla Hash del Servidor DNS (Con Estado de Cada Posición)

El reporte debe mostrar todas las posiciones de la tabla (índices 0 a m − 1), indicando para cada una si está VACÍA, OCUPADA o BORRADA, el hostname y la IP almacenados, junto con un panel de métricas: tamaño m, número de registros n, factor de carga actual y funciones hash utilizadas.

Tabla Hash DNS

m = 13 (primo)

n = 7 registros

α = n/m = 0.54

α máx = 0.70

h1(k) = k mod 13

h2(k) = 1 + (k mod 11)

Índice	Hostname (clave)	Dirección IP	Estado
0	api.cloudgrid.gt	10.0.1.40	OCUPADO
1			VACÍO
2	auth.cloudgrid.gt	10.0.1.20	OCUPADO
3	db-master.cloudgrid.gt	10.0.1.60	OCUPADO
4	legacy.cloudgrid.gt	—	BORRADO
5			VACÍO
6	cache.cloudgrid.gt	10.0.2.30	OCUPADO
7	dns-gw.cloudgrid.gt	10.0.2.10	OCUPADO (colisión, 2 sondeos)
8			VACÍO
9	worker01.cloudgrid.gt	10.0.2.50	OCUPADO
10			VACÍO
11	mail.cloudgrid.gt	10.0.3.15	OCUPADO
12			VACÍO

Figura 1: Tabla Hash DNS con direccionamiento abierto, lápidas de borrado y métricas de factor de carga.

Reporte 2: Matriz Dispersa de Tráfico entre Data Centers

Representación ortogonal con el nodo raíz, las cabeceras de filas (DC origen) y de columnas (DC destino), y únicamente los nodos con tráfico registrado, enlazados en ambas direcciones de forma horizontal y vertical. Cada nodo debe mostrar el volumen en GB y el número de trabajos.

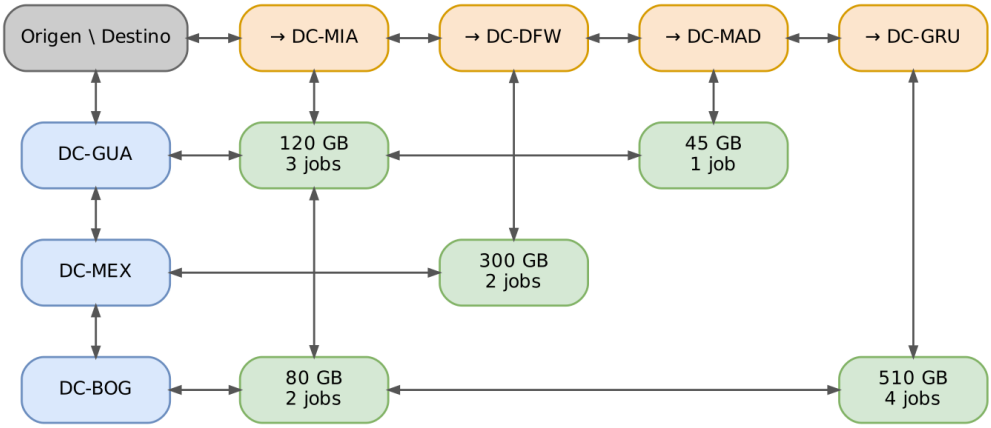
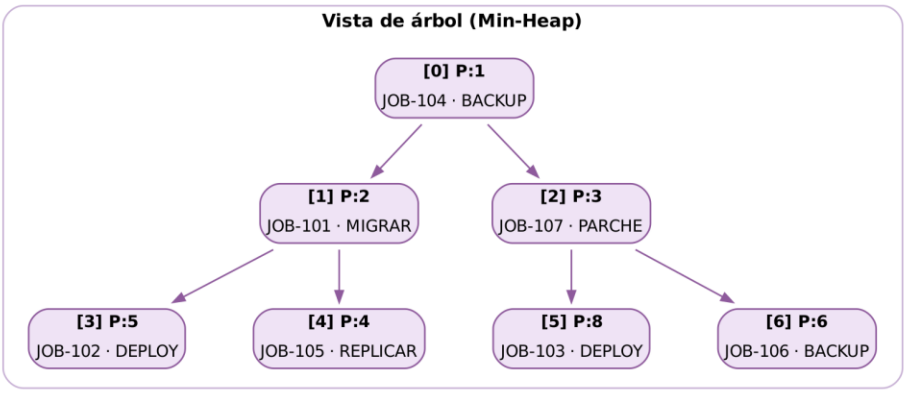


Figura 2: Matriz Dispersa ortogonal con cabeceras de origen y destino y celdas de tráfico acumulado.

Reporte 3: Min-Heap del Planificador de Trabajos (Vista de Árbol y de Arreglo)

El reporte debe mostrar simultáneamente la vista de árbol binario completo y el arreglo subyacente con sus índices, de modo que se evidencie la relación padre(i) = (i − 1)/2, hijo izquierdo 2i + 1 e hijo derecho 2i + 2. Cada nodo debe mostrar su índice, prioridad, ID y tipo de trabajo.



i	0	1	2	3	4	5	6
P	1	2	3	5	4	8	6
Job	104	101	107	102	105	103	106

Arreglo: padre(i) = (i-1)/2 · izq(i) = 2i+1 · der(i) = 2i+2

Figura 3: Min-Heap del planificador con su representación equivalente en arreglo.

Reporte 4: Grafo de la Red Backbone (Con Niveles BFS y Lista de Adyacencia)

Debe generarse la vista del grafo no dirigido con la latencia como etiqueta de cada arista. Al ejecutar un BFS, cada vértice debe colorearse según su nivel (número de saltos desde el

origen) y la ruta encontrada debe resaltarse. Adicionalmente, se debe generar la representación interna de la lista de adyacencia, evidenciando la estructura real en memoria.

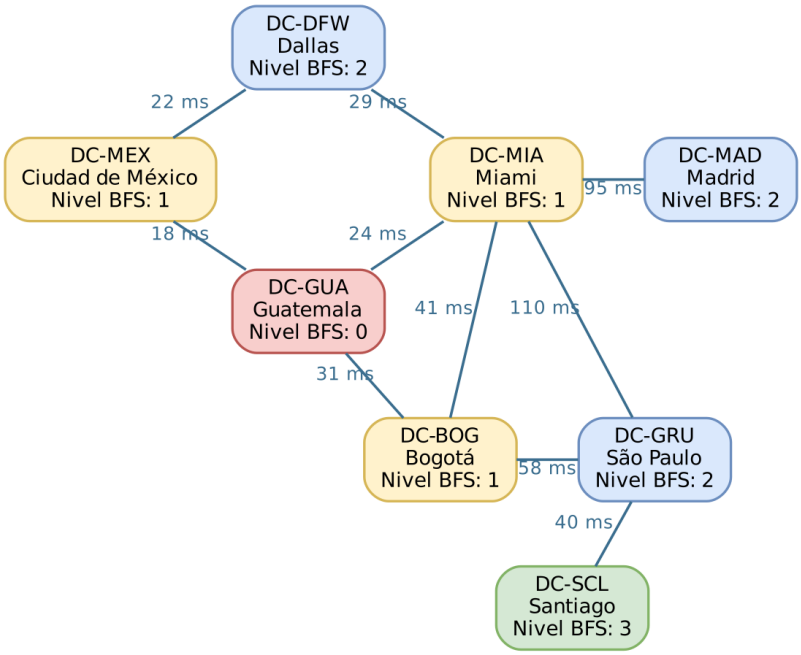


Figura 4: Grafo de la red backbone coloreado por niveles de un BFS iniciado en DC-GUA.

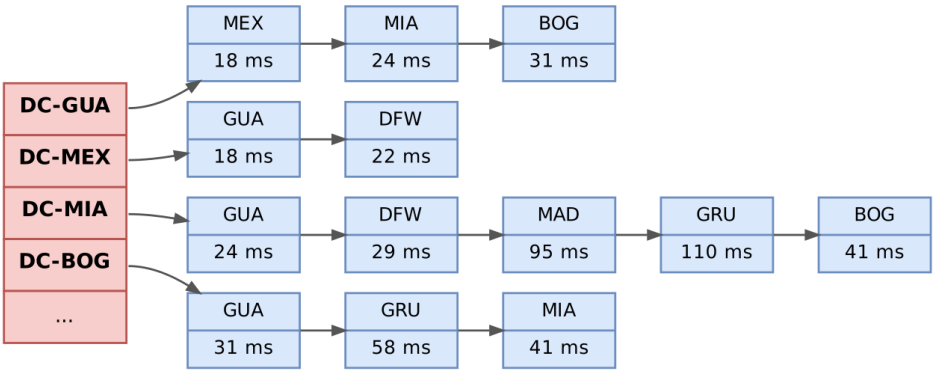


Figura 5: Lista de adyacencia de la red backbone (vista parcial).

4.2 Alcance del proyecto

Las estructuras de datos deben ser desarrolladas completamente por los estudiantes sin utilizar librerías o módulos predefinidos en C++ que implementen directamente estas estructuras (std::unordered_map, std::map, std::priority_queue, std::queue, std::stack, std::vector, std::list, entre otros). Toda la lógica de dispersión, sondeo, enlazado, reacomodo del heap y recorridos debe ser manual; las colas y pilas auxiliares que requieren BFS y DFS también deben ser implementadas por el estudiante.

Los reportes gráficos son esenciales para la evaluación. Si no se presenta el reporte de alguna estructura, se anularán los puntos correspondientes. El código debe ser original, individual y debidamente comentado

4.2 Requerimiento técnicos

Realizar la aplicación en c++

El sistema se desarrollará completamente en c++, implementando una aplicación con interfaz que gestione todas las funcionalidades descritas. Se utilizarán las capacidades nativas de c++ para el manejo de archivos CSV y procesamiento de texto. El código deberá ser modular, con funciones claramente definidas para cada operación del sistema.

Realizar los reportes en Graphviz

Todos los reportes gráficos deberán generarse utilizando exclusivamente Graphviz. El sistema c++ creará archivos DOT con la descripción de los grafos y ejecutará los comandos necesarios para generar las imágenes en formato PNG o PDF. Los reportes se actualizarán automáticamente cada vez que se modifiquen las estructuras de datos, permitiendo una visualización en tiempo real del estado del sistema.

5. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

5.1 Proyectos Individuales

Los proyectos individuales permiten a los estudiantes desarrollar autonomía y responsabilidad sobre su propio trabajo. En este tipo de proyectos, cada estudiante debe gestionar su tiempo, investigar, planificar y ejecutar todas las fases del proyecto de manera independiente.

5.1.1 Autogestión del Tiempo

Los estudiantes deben crear un cronograma personal para cumplir con los plazos

establecidos. Esto les ayuda a mejorar su disciplina y capacidad de priorización, habilidades esenciales en cualquier entorno profesional.

5.1.2 Responsabilidad y Compromiso

En un proyecto individual, el estudiante asume la totalidad de las responsabilidades, desde la investigación hasta la entrega final. Esto fomenta el sentido de compromiso y permite una mayor personalización en la solución del problema planteado.

5.1.3 Resolución de Problemas

Trabajar de manera independiente impulsa a los estudiantes a buscar soluciones de forma creativa y a enfrentarse a los desafíos sin depender del apoyo constante de otros. Esto fortalece su capacidad para resolver problemas de manera autónoma.

5.1.4 Reflexión Personal

Al concluir el proyecto, el estudiante realiza una autoevaluación, reflexionando sobre sus decisiones, lo aprendido y las áreas en las que podría mejorar. Esta práctica promueve el desarrollo continuo y el autoaprendizaje.

5.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Implementación de estructuras de datos	**40**	
Tabla Hash (Direccionamiento Abierto) para Servidor DNS	10	
Matriz Dispersa Ortogonal para Tráfico entre Data Centers	9	
Montículo Binario Mínimo (Min-Heap) para Planificador de Trabajos	9	
Grafo (Lista de Adyacencia) con Recorridos BFS / DFS para Red Backbone	12	
2. Funcionalidades del sistema	**31**	
Funciones del Módulo Administrador (Carga CSV, Red, DNS, Heap, Recorridos, Tráfico)	19	
Funciones del Módulo Usuario DevOps (DNS, Trabajos, Rutas, Tráfico)	12	
3. Reporte gráficos con Graphviz	**18**	
Reporte de Tabla Hash DNS	4	
Reporte de Matriz Dispersa de Tráfico	5	
Reporte de Min-Heap (Árbol y Arreglo)	4	
Reporte de Grafo (Niveles BFS y Lista de Adyacencia)	5	
4. Manual técnico y preguntas	**11**	
TOTAL	**100**	

5.3 Observaciones

- El lenguaje de programación para esta fase será c++.
- La herramienta de desarrollo de reportes gráficos será exclusivamente Graphviz.
- La entrega se realizará por medio de Github. El nombre del repositorio debe ser EDD_1S2027_PY_#carnet (Ej: EDD_1S2027_PY_202102771), donde se creará una carpeta con el nombre EDD_Fase2.
- Por medio de UEDI se hará entrega del link del repositorio.
- Deberá agregar a su respectivo auxiliar a su reporte como colaborador:
    - user → Sección B
    - user → Sección C
    - user → sección A
- Los repositorios deben mantenerse en privado para evitar copias de código entre estudiantes.
- Se debe realizar el manual técnico en formato Markdown dentro de una carpeta llamada documentación, dentro de la carpeta de la respectiva fase, explicando la implementación de cada estructura, las decisiones de diseño tomadas, la complejidad de cada operación y por qué la ruta de menor número de saltos (BFS) no siempre coincide con la de menor latencia.
- Todas las dudas durante el proceso de desarrollo deberán realizarse por medio de los foros de UEDI de la respectiva sección.
- Las copias detectadas serán penalizadas con calificación de 0 puntos y reportadas a la Escuela de Ciencias y Sistemas.
- Se debe de realizar commits frecuentes en Github para demostrar el avance progresivo del proyecto.
- El código debe estar debidamente comentado y seguir buenas prácticas de programación.
- Se deben incluir los archivos de prueba CSV (Red_Backbone, Registros_DNS, Trabajos e Historial_Trafico) con datos de ejemplo para demostrar la funcionalidad de carga masiva.

FECHA DE ENTREGA: xx de xx del 2027